

Distributed Machine Learning

Assignment 2

Implementation and Performance Analysis of the Federated Averaging (FedAvg)
Algorithm for Image Classification

Group Number: 55

Group 55 - Team Members

S. No.	Name	ID Number
1	Adurti Sai Venkatesh	2024ac05968
2	Akkireddy Pranith	2024ac05473
3	D. Mallikarjuna Reddy	2024ac05653

Course: Distributed Machine Learning

Course Code: AIMLCZG515

Topic: Federated Learning and Federated Averaging (FedAvg)

Dataset: MNIST Handwritten Digits

Submitted as part of the requirements of the Distributed Machine Learning course.

Table of Contents

Abstract	3
1. Problem Statement and Objectives	4
1.1 Problem Statement	4
1.2 Objectives	4
1.3 Experimental Configuration	4
2. Dataset Description and Methodology	5
2.1 Dataset	5
2.2 Data Preprocessing	5
2.3 Tools and Libraries	5
3. Client Data Partitioning Strategy	6
3.1 IID Partitioning	6
3.2 Non-IID Partitioning	6
4. Local Model Architecture	8
5. Federated Averaging Workflow	9
5.1 Algorithm	9
5.2 Implementation Assumptions	9
6. Source Code	10
7. Execution Results	13
7.1 Main FedAvg Run: IID, Local Epochs = 2	13
7.2 Experiment 1: Local-Epoch Comparison	14
7.3 Experiment 2: IID versus Non-IID	15
7.4 Client Drift	15
7.5 Class-Wise Evaluation	16
8. Consolidated Results	17
9. Analysis and Discussion	17
9.1 Critical Analysis	18
10. Limitations and Practical Considerations	18
11. Conclusion	18
12. References	20

Abstract

This report presents the manual implementation and performance evaluation of the Federated Averaging (FedAvg) algorithm for MNIST image classification. Five simulated clients collaboratively train a lightweight convolutional neural network without sharing their local image data. The study evaluates both IID and label-shard Non-IID client partitions, executes ten communication rounds, and compares local training epochs of 1, 2 and 5 under controlled initialization and preprocessing conditions.

The principal IID experiment, using two local epochs, achieved 99.28% global test accuracy and 0.0254 loss after ten communication rounds. Increasing local computation accelerated early convergence, but five local epochs produced only 99.26% final accuracy and greater late-round loss fluctuation, demonstrating diminishing returns. Under the Non-IID split, final accuracy reached 78.76%, creating a 20.52 percentage-point gap relative to the equivalent IID experiment. Class-wise analysis identified digit 5 as the most affected class, with its F1-score declining from 0.990 to 0.317.

The results show that standard FedAvg is highly effective when client objectives are aligned, but severe statistical heterogeneity causes local specialization, conflicting updates and weak global generalization. FedProx is identified as a suitable extension for controlling client drift under heterogeneous data.

Key Experimental Results

Configuration	Final Accuracy	Final Loss
IID - 1 local epoch	99.10%	0.0286
IID - 2 local epochs	99.28%	0.0254
IID - 5 local epochs	99.26%	0.0317
Non-IID - 2 local epochs	78.76%	0.6967

Implementation constraint: FedAvg was coded from scratch. No federated-learning framework such as TensorFlow Federated, Flower, FedML or PySyft was used.

1. Problem Statement and Objectives

1.1 Problem Statement

A consortium of hospitals requires a shared image-classification model for disease diagnosis. Regulatory constraints prevent the hospitals from transferring patient data to a central server, so each hospital trains a model locally on its own data and shares only the resulting model parameters. A central coordinator aggregates these parameters into a single global model without ever accessing the underlying patient data.

This assignment implements that scenario using the Federated Averaging (FedAvg) algorithm. The MNIST handwritten-digit dataset is used as the required image-classification benchmark, and five simulated hospitals (referred to as clients) are constructed from it. FedAvg is implemented manually, without using any federated-learning framework such as TensorFlow Federated, Flower, FedML or PySyft.

1.2 Objectives

- Prepare the MNIST dataset and construct five IID and five Non-IID client partitions.
- Develop a compact convolutional neural network and train it locally at each client.
- Implement sample-count-weighted Federated Averaging manually, with no FL framework.
- Execute ten communication rounds and record client-level and global-level metrics.
- Compare local training epochs of 1, 2 and 5 under identical experimental conditions.
- Compare IID and Non-IID convergence behaviour, accuracy, loss and class-wise performance.
- Analyse client heterogeneity critically and identify an improvement over standard FedAvg.

1.3 Experimental Configuration

Component	Setting
Simulated clients	5
Communication rounds	10
Local batch size	32
Primary local-epoch setting	2
Local-epoch comparison values	1, 2, 5
Model	Lightweight convolutional neural network (CNN)
Optimizer	Adam
Loss function	Sparse categorical cross-entropy
Aggregation rule	Manual sample-count-weighted Federated Averaging
Evaluation set	Centrally held MNIST test set (10,000 images)

2. Dataset Description and Methodology

2.1 Dataset

The MNIST Handwritten Digits dataset is used, consisting of 60,000 training images and 10,000 testing images, each a 28x28 grayscale image labelled with one of ten digit classes (0-9). The dataset was loaded using the TensorFlow/Keras dataset loader. Structural validation confirmed that both the training and testing arrays contain no missing values, no infinite values, no blank images and no out-of-range pixel intensities, and that every image carries a valid label in the range 0-9.

Class frequencies in the training set range from 5,421 samples (digit 5) to 6,742 samples (digit 1), corresponding to an imbalance ratio of 1.24, which is close to the ratio observed in the testing set (1.27). The dataset is therefore treated as approximately class-balanced, and no class re-weighting or resampling is applied.

2.2 Data Preprocessing

Pixel intensities were rescaled from the original 0-255 integer range to the floating-point range 0.0-1.0 using min-max normalisation, and a channel dimension was appended so that every image has the shape 28x28x1, as required by the Conv2D input layer. This preprocessing pipeline is applied once, centrally, and identically to every training and testing sample before any client partitioning takes place, which ensures that every client's local weight updates remain in a numerically compatible parameter space for FedAvg aggregation.

2.3 Tools and Libraries

Library	Role in this implementation
TensorFlow / Keras	Model definition, local training, dataset loading
NumPy	Array operations, partition construction, weighted aggregation
Pandas	Result logging and tabular summarisation
Matplotlib	Accuracy/loss curves, class-distribution and confusion-matrix plots
scikit-learn	Confusion matrices and classification reports

The implementation was executed in a Kubeflow notebook environment with a single NVIDIA A100 GPU. Random seeds were fixed (seed = 42) across NumPy, Python's random module and TensorFlow to make dataset partitioning, model initialisation and reported results reproducible.

3. Client Data Partitioning Strategy

3.1 IID Partitioning

The 60,000 training images are shuffled using a fixed random seed and split equally into five clients of 12,000 images each. Because the shuffle is uniform over the whole training set, every client receives approximately the same proportion of every digit class. The table below reports the number of training samples assigned to each client, broken down by digit class, and confirms that all 60,000 samples were assigned exactly once across the five clients.

Client	Total	D0	D1	D2	D3	D4	D5	D6	D7	D8	D9
Client_1	12000	1211	1369	1189	1267	1168	1063	1179	1222	1152	1180
Client_2	12000	1153	1331	1208	1249	1190	1137	1164	1219	1163	1186
Client_3	12000	1145	1334	1166	1196	1195	1110	1195	1235	1227	1197
Client_4	12000	1202	1333	1203	1215	1155	1046	1196	1326	1151	1173
Client_5	12000	1212	1375	1192	1204	1134	1065	1184	1263	1158	1213

Across all ten digit classes, the per-client share stays within a narrow band (class standard deviation of 0.10-0.33 percentage points around the expected 10% share; see the class-wise variation obtained during exploratory analysis). This confirms that the partition is a genuine IID split rather than one that merely balances totals while skewing class content.

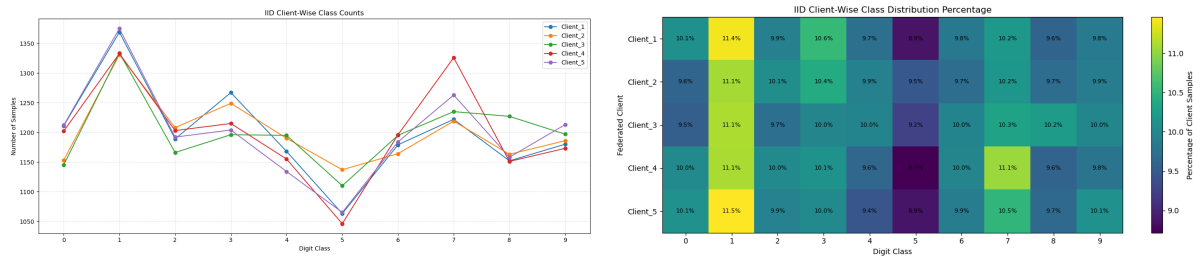


Figure 3.1: Per-client class counts (left) and class-share heatmap (right) under the IID partition, showing near-identical distributions across clients.

3.2 Non-IID Partitioning

The Non-IID partition uses a label-shard strategy. The 60,000 training indices are first sorted by digit label and divided into ten equal shards; each client is then assigned two shards, so every client's data is dominated by at most two to four digit classes rather than a representative mixture of all ten. This simulates a realistic federated setting in which different hospitals hold patients with different, non-representative case distributions.

Client	Shards	Total	D0	D1	D2	D3	D4	D5	D6	D7	D8	D9
Client_1	[5, 6]	12000	0	0	0	0	596	5421	5918	65	0	0
Client_2	[0, 7]	12000	5923	77	0	0	0	0	0	6000	0	0
Client_3	[3, 2]	12000	0	665	5958	5377	0	0	0	0	0	0
Client_4	[4, 9]	12000	0	0	0	754	5246	0	0	0	51	5949
Client_5	[1, 8]	12000	0	6000	0	0	0	0	0	200	5800	0

On average, 49.7% of a client's 12,000 samples belong to a single dominant class, and every client observes only three to four of the ten possible classes (dominant classes: Client_1 = digit 6, Client_2 = digit 7, Client_3 = digit 2, Client_4 = digit 9, Client_5 = digit 1). Each client still holds exactly 12,000 samples, matching the IID partition size, so that any accuracy difference between the IID and Non-IID experiments is attributable to label distribution rather than to dataset size.

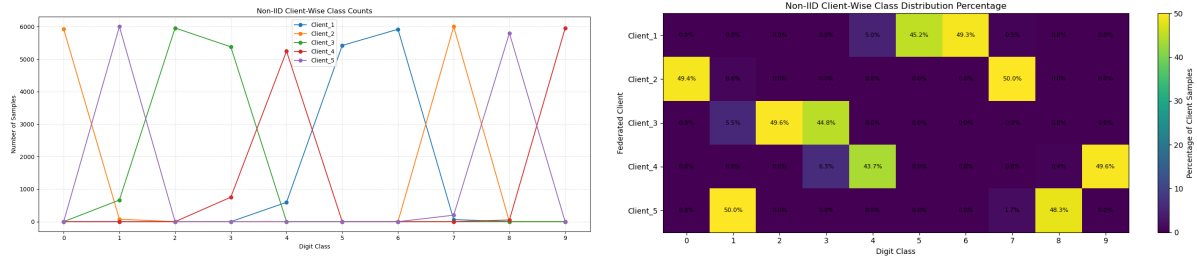


Figure 3.2: Per-client class counts (left) and class-share heatmap (right) under the Non-IID partition, showing severe label concentration.

4. Local Model Architecture

A compact convolutional neural network (CNN) is used as the classification model at every client and at the global aggregator. A CNN was selected over a Multi-Layer Perceptron because exploratory analysis of MNIST showed systematic, class-dependent spatial structure (stroke shape, stroke density and foreground footprint varying by digit); a CNN exploits this spatial structure directly through local receptive fields and pooling, whereas an MLP on flattened pixels treats every pixel as an unordered, independent feature. The architecture is intentionally lightweight because it must be instantiated and trained repeatedly: once per client, per communication round, across four separate experiments.

Layer	Configuration	Parameters
Input	28 x 28 x 1	-
Conv2D	32 filters, 3x3 kernel, ReLU	9,248
MaxPooling2D	2x2 pool	0
Conv2D	64 filters, 3x3 kernel, ReLU	18,496
MaxPooling2D	2x2 pool	0
Flatten	-	0
Dense	64 units, ReLU	92,224
Dense (output)	10 units, Softmax	650

The model has 121,930 trainable parameters in total. It is compiled with the Adam optimizer and sparse categorical cross-entropy loss, and accuracy is tracked as the evaluation metric, matching the assignment's batch-size and loss-function requirements.

5. Federated Averaging Workflow

Federated Averaging is implemented manually, with no federated-learning orchestration library. The algorithm follows the standard four-stage communication round: weight distribution, local training, weighted aggregation, and global evaluation.

5.1 Algorithm

```
Initialise global model weights  $w_0$ 

For communication round  $t = 1 \dots 10$ :
  For each client  $k = 1 \dots 5$ :
    Copy current global weights  $w_t$  to a fresh local model
    Train locally for  $E$  epochs with batch size 32 on client  $k$ 's private data
    Return locally trained weights  $w_{(t+1)}^k$  and local sample count  $n_k$ 

  Compute total sample count  $N = \sum_k n_k$ 
  Aggregate manually (no FedAvg library):
     $w_{(t+1)} = \sum_k (n_k / N) * w_{(t+1)}^k$ 

  Evaluate  $w_{(t+1)}$  on the central MNIST test set
  Record global loss, global accuracy, and every client's local metrics
```

5.2 Implementation Assumptions

- Five federated clients are simulated within a single notebook environment; all clients participate in every communication round.
- Client communication is simulated; network latency, failures and dropouts are not modelled.
- Only updated model weights and training metrics leave a client; the complete MNIST test set is retained centrally, purely for global evaluation.
- All clients use an identical CNN architecture, optimizer, batch size and preprocessing pipeline, and start every experiment from the same reference weights.
- Federated Averaging is implemented manually using sample-count-weighted, layer-wise aggregation of model parameters.

Federated Communication-Round Flow

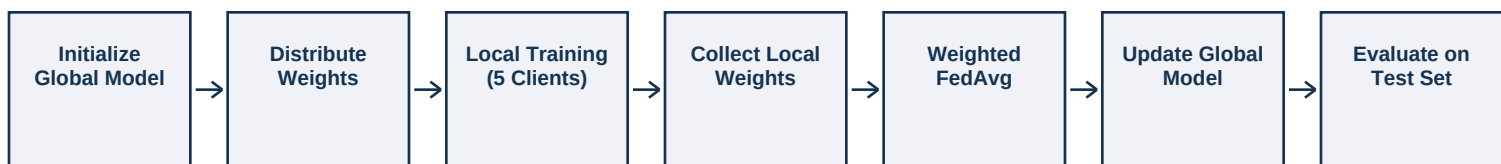


Figure 5.1: Synchronous full-participation workflow repeated for each of the ten communication rounds.

6. Source Code

This section presents the complete, executable implementation of client data partitioning, the CNN model, local client training, manual Federated Averaging, global evaluation and the orchestration function that ties these components into a full federated-learning run. All code below executed successfully in the accompanying Jupyter notebook.

6.1 IID Client Partitioning

```
# Create five IID client partitions
NUM_CLIENTS = 5
rng = np.random.default_rng(SEED)
all_indices = np.arange(len(x_train_processed))
shuffled_indices = rng.permutation(all_indices)
iid_index_splits = np.array_split(shuffled_indices, NUM_CLIENTS)

iid_clients = {}
for client_id, client_indices in enumerate(iid_index_splits, start=1):
    iid_clients[f"Client_{client_id}"] = {
        "x": x_train_processed[client_indices],
        "y": y_train[client_indices],
        "indices": client_indices
    }

# Display the number of training samples assigned to each client, by digit
iid_partition_summary = pd.DataFrame([
    {"Client": name, "Total Samples": len(data["y"]),
     **{f"Digit {d}": np.bincount(data["y"], minlength=10)[d] for d in range(10)}}
    for name, data in iid_clients.items()
])
display(iid_partition_summary)
```

6.2 Non-IID Client Partitioning

```
# Create five Non-IID client partitions using label shards
NUM_SHARDS, SHARDS_PER_CLIENT = 10, 2
sorted_indices = np.argsort(y_train) # group indices by label
non_iid_shards = np.array_split(sorted_indices, NUM_SHARDS) # 10 equal label-sorted shards
rng = np.random.default_rng(SEED)
shard_order = rng.permutation(NUM_SHARDS)

non_iid_clients = {}
for client_index in range(NUM_CLIENTS):
    assigned_shard_ids = shard_order[
        client_index * SHARDS_PER_CLIENT: (client_index + 1) * SHARDS_PER_CLIENT
    ]
    client_indices = np.concatenate([non_iid_shards[s] for s in assigned_shard_ids])
    non_iid_clients[f"Client_{client_index + 1}"] = {
        "x": x_train_processed[client_indices], "y": y_train[client_indices],
        "indices": client_indices, "shards": assigned_shard_ids
    }

# Display the number of training samples assigned to each client, by digit
non_iid_partition_summary = pd.DataFrame([
    {"Client": name, "Assigned Shards": data["shards"].tolist(), "Total Samples": len(data["y"]),
     **{f"Digit {d}": np.bincount(data["y"], minlength=10)[d] for d in range(10)}}
    for name, data in non_iid_clients.items()
])
display(non_iid_partition_summary)
```

6.3 CNN Model Definition

```
def create_cnn_model():
    """Create and compile a lightweight CNN for MNIST classification.
    A fresh instance is used for the global model and for every local client."""
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(28, 28, 1)),
        tf.keras.layers.Conv2D(filters=32, kernel_size=(3, 3), activation="relu"),
        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
        tf.keras.layers.Conv2D(filters=64, kernel_size=(3, 3), activation="relu"),
        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(units=64, activation="relu"),
        tf.keras.layers.Dense(units=10, activation="softmax")
    ])
    model.compile(
        optimizer=tf.keras.optimizers.Adam(),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model

global_model = create_cnn_model()
global_model.summary()
```

6.4 Local Client Training

```
def train_local_client(global_weights, client_data, local_epochs=2, batch_size=32, verbose=0):
    """
    Train a fresh local CNN using one client's private dataset.
    Receives the current global weights, trains locally for `local_epochs`
    epochs at the specified batch size, and returns the updated local
    weights plus local loss/accuracy/sample-count metrics for FedAvg.
    """
    local_model = create_cnn_model()
    local_model.set_weights(global_weights)

    history = local_model.fit(
        client_data["x"], client_data["y"],
        epochs=local_epochs, batch_size=batch_size, shuffle=True, verbose=verbose
    )

    metrics = {
        "local_loss": float(history.history["loss"][-1]),
        "local_accuracy": float(history.history["accuracy"][-1]),
        "sample_count": len(client_data["y"])
    }
    updated_weights = local_model.get_weights()

    del local_model
    tf.keras.backend.clear_session()
    return updated_weights, metrics
```

6.5 Manual Federated Averaging

```
def federated_average(client_weights, client_sample_counts):
    """
    Perform weighted Federated Averaging across client model weights.
    Each client's weight tensors are scaled by (its sample count / total
    samples) and summed layer-by-layer -- implemented manually, with no
    federated-learning library.
    """
    if len(client_weights) == 0:
        raise ValueError("client_weights cannot be empty.")
    if len(client_weights) != len(client_sample_counts):
        raise ValueError("client_weights and client_sample_counts must have the same length.")
    if any(n <= 0 for n in client_sample_counts):
        raise ValueError("All client sample counts must be positive.")

    total_samples = np.sum(client_sample_counts)
    averaged_weights = [np.zeros_like(w, dtype=np.float32) for w in client_weights[0]]

    for weights, sample_count in zip(client_weights, client_sample_counts):
        client_weight = sample_count / total_samples
        for layer_index, layer_weights in enumerate(weights):
            averaged_weights[layer_index] += client_weight * layer_weights

    return averaged_weights
```

6.6 Global Model Evaluation

```
def evaluate_global_model(model, x_test_data, y_test_data):
    """Evaluate the current global model on the common, centrally held MNIST
    test set and return the global loss and global accuracy."""
    global_loss, global_accuracy = model.evaluate(x_test_data, y_test_data, verbose=0)
    return {"global_loss": float(global_loss), "global_accuracy": float(global_accuracy)}
```

6.7 Federated Training Orchestration

```
def run_federated_training(
    client_datasets, communication_rounds=10, local_epochs=2, batch_size=32,
    experiment_name="Federated Experiment", initial_weights=None,
    verbose_local_training=0
):
    """
    Run manual Federated Averaging across simulated clients.
    Returns the final global model, the round-wise global history
    (loss/accuracy per communication round) and the client-wise
    local history (loss/accuracy per client per round).
    """
    if communication_rounds <= 0 or local_epochs <= 0 or batch_size <= 0:
        raise ValueError("communication_rounds, local_epochs and batch_size must be positive.")
    if not client_datasets:
        raise ValueError("client_datasets cannot be empty.")

    trained_global_model = create_cnn_model()
    if initial_weights is not None:
        trained_global_model.set_weights([w.copy() for w in initial_weights])

    round_records, client_records = [], []
    print(experiment_name.upper())
```

```

print(f"Rounds={communication_rounds} LocalEpochs={local_epochs} BatchSize={batch_size} "
      f"Clients={len(client_datasets)}")

for round_number in range(1, communication_rounds + 1):
    current_global_weights = trained_global_model.get_weights()
    collected_client_weights, client_sample_counts = [], []
    print(f"\nCommunication Round {round_number}/{communication_rounds}")

    # ---- Weight distribution + local training at every client ----
    for client_name, client_data in client_datasets.items():
        updated_weights, local_metrics = train_local_client(
            global_weights=current_global_weights, client_data=client_data,
            local_epochs=local_epochs, batch_size=batch_size,
            verbose=verbose_local_training
        )
        collected_client_weights.append(updated_weights)
        client_sample_counts.append(local_metrics["sample_count"])
        client_records.append({
            "Experiment": experiment_name, "Round": round_number, "Client": client_name,
            "Local Epochs": local_epochs, "Sample Count": local_metrics["sample_count"],
            "Local Loss": local_metrics["local_loss"],
            "Local Accuracy": local_metrics["local_accuracy"]
        })
    print(f"{client_name:<10} | Samples: {local_metrics['sample_count']:5d} | "
          f"Local Loss: {local_metrics['local_loss']:.4f} | "
          f"Local Accuracy: {local_metrics['local_accuracy'] * 100:.2f}%")

    # ---- Manual weighted Federated Averaging (no FL framework) ----
    averaged_weights = federated_average(collected_client_weights, client_sample_counts)
    trained_global_model.set_weights(averaged_weights)

    # ---- Evaluate the updated global model after this round ----
    global_metrics = evaluate_global_model(trained_global_model, x_test_processed, y_test)
    round_records.append({
        "Experiment": experiment_name, "Round": round_number, "Local Epochs": local_epochs,
        "Global Loss": global_metrics["global_loss"],
        "Global Accuracy": global_metrics["global_accuracy"]
    })
    print(f"Global Loss: {global_metrics['global_loss']:.4f} "
          f"Global Accuracy: {global_metrics['global_accuracy'] * 100:.2f}%")

    round_history = pd.DataFrame(round_records)
    client_history = pd.DataFrame(client_records)
    return trained_global_model, round_history, client_history

```

6.8 Experiment Invocation (IID, Local Epochs = 2)

```

iid_model_epoch_2, iid_round_history_epoch_2, iid_client_history_epoch_2 = (
    run_federated_training(
        client_datasets=iid_clients,
        communication_rounds=10,
        local_epochs=2,
        batch_size=32,
        experiment_name="IID FedAvg - Local Epoch 2",
        initial_weights=REFERENCE_INITIAL_WEIGHTS,
        verbose_local_training=0
    )
)

```

7. Execution Results

7.1 Main FedAvg Run (IID, Local Epochs = 2)

The primary experiment initialises the global model from a shared reference set of weights and runs ten communication rounds over the five IID clients, with each client training locally for two epochs at a batch size of 32. The table below reports the global loss and global accuracy recorded after every communication round.

Round	Global Loss	Global Accuracy (%)
1	0.0840	97.59
2	0.0416	98.58
3	0.0334	98.82
4	0.0296	99.02
5	0.0289	99.10
6	0.0273	99.12
7	0.0274	99.10
8	0.0288	99.14
9	0.0307	99.09
10	0.0254	99.28

Global accuracy rises from 97.59% in Round 1 to 99.28% in Round 10, and the lowest global loss (0.0254) also occurs in Round 10. The local training accuracy of every individual client, before aggregation, is reported below for Round 1 and Round 10.

Client	Round 1 Local Accuracy (%)	Round 10 Local Accuracy (%)
Client_1	97.09	99.63
Client_2	96.74	99.73
Client_3	97.03	99.76
Client_4	96.89	99.76
Client_5	96.45	99.84

In Round 1, every individual client reaches a local accuracy between 96.45% and 97.09%, yet the aggregated global model already reaches 97.59%, exceeding every individual client. This demonstrates the core mechanism of FedAvg: averaging several independently-trained, identically-initialised models in weight space acts similarly to model ensembling, smoothing out the idiosyncratic error of any single client.

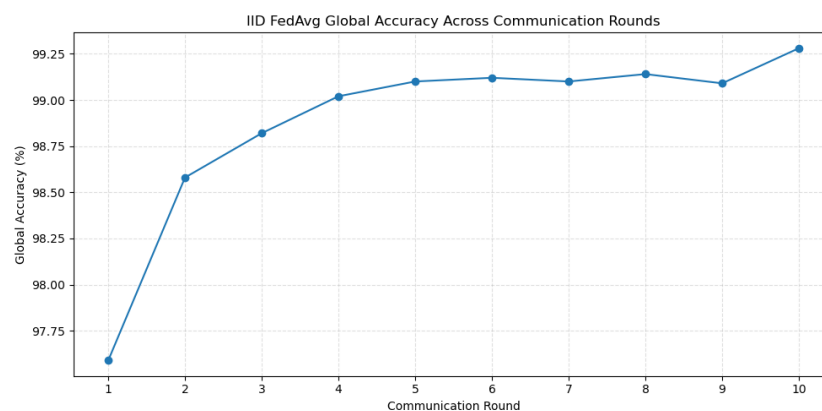


Figure 7.1: Global accuracy across ten communication rounds (IID, local epochs = 2).

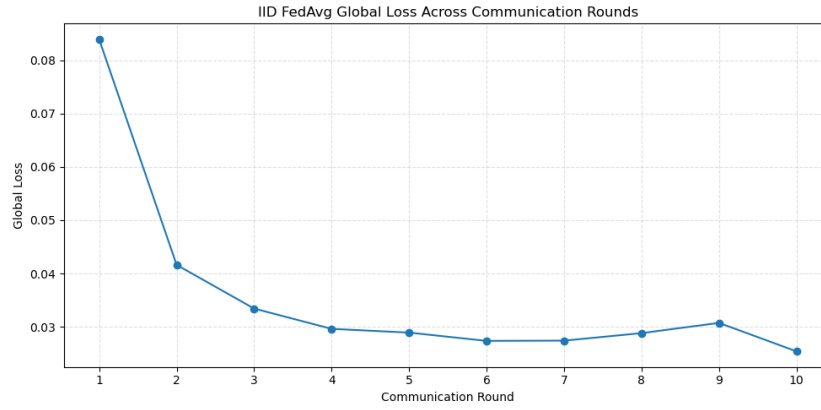


Figure 7.2: Global loss across ten communication rounds (IID, local epochs = 2).

7.2 Experiment 1: Local-Epoch Comparison

Local epochs of 1, 2 and 5 were compared under the same IID partition, reference initialisation, batch size and ten communication rounds, isolating the effect of local computation per round.

Local Epochs	Round 1 Accuracy	Final Accuracy	Best Accuracy	Final Loss	Lowest Loss
1	96.57%	99.10%	99.17% (Round 9)	0.0286	0.0274 (Round 9)
2	97.59%	99.28%	99.28% (Round 10)	0.0254	0.0254 (Round 10)
5	98.36%	99.26%	99.30% (Round 9)	0.0317	0.0279 (Round 5)

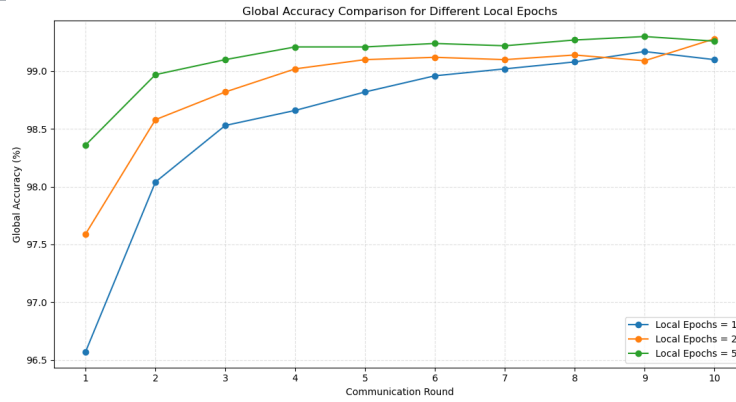


Figure 7.3: Global accuracy for local epochs 1, 2 and 5 (IID partition).

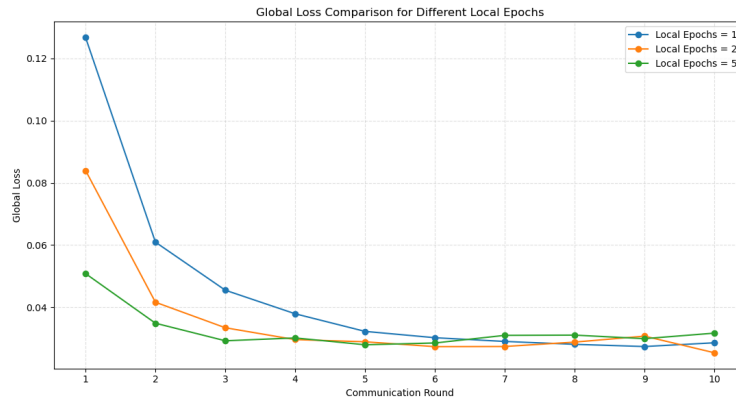


Figure 7.4: Global loss for local epochs 1, 2 and 5 (IID partition).

Final global accuracy differs by only 0.22 percentage points across a five-fold range of local computation (99.10% at 1 epoch versus 99.28% at 2 epochs versus 99.26% at 5 epochs). Five local epochs produce the highest single-round accuracy (99.30% at Round 9) but not the best final result, and their loss curve fluctuates more in later rounds. Under this IID setting, additional local epochs mainly accelerate early-round convergence rather than improving the final outcome, and

two local epochs already operate close to the point of diminishing returns.

7.3 Experiment 2: IID versus Non-IID Comparison

The Non-IID experiment repeats the identical configuration (reference initialisation, two local epochs, batch size 32, ten rounds) on the label-shard Non-IID partition described in Section 3.2, isolating the effect of client data heterogeneity.

Distribution	Round 1 Accuracy	Final Accuracy	Best Accuracy	Final Loss	Lowest Loss
IID	97.59%	99.28%	99.28%	0.0254	0.0254
Non-IID	41.06%	78.76%	78.76%	0.6967	0.6967

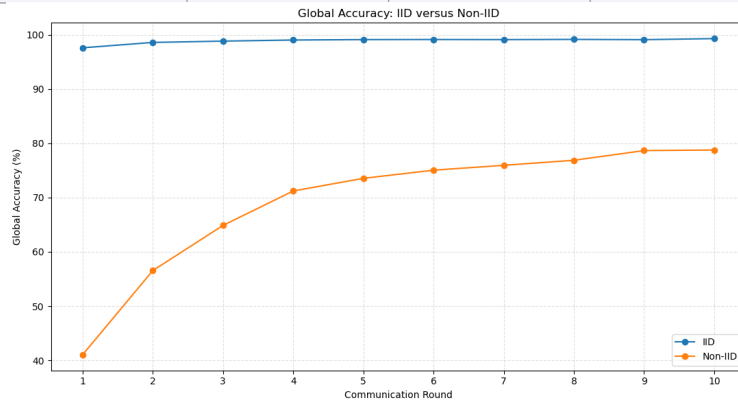


Figure 7.5: Global accuracy, IID versus Non-IID partitions.

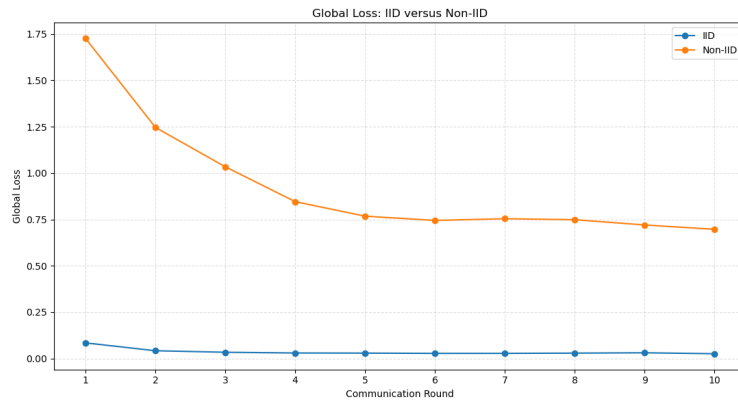


Figure 7.6: Global loss, IID versus Non-IID partitions.

The Non-IID global model improves from 41.06% to 78.76% across ten rounds but remains 20.52 percentage points below the equivalent IID model, and its final loss (0.6967) is more than 27 times higher than the IID final loss (0.0254). This gap arises entirely from how the same 60,000 training images are distributed across the five clients, since every other experimental condition is identical.

7.4 Client Drift: Dispersion of Local Accuracy

Client drift is quantified directly as the round-wise standard deviation of the five clients' local training accuracy, computed separately for the IID and Non-IID runs.

Round	IID Std Dev (pp)	Non-IID Std Dev (pp)
1	0.257	0.354
2	0.025	0.303
3	0.061	0.262
4	0.083	0.231
5	0.095	0.172
6	0.049	0.145

Round	IID Std Dev (pp)	Non-IID Std Dev (pp)
7	0.068	0.117
8	0.065	0.118
9	0.032	0.106
10	0.075	0.122

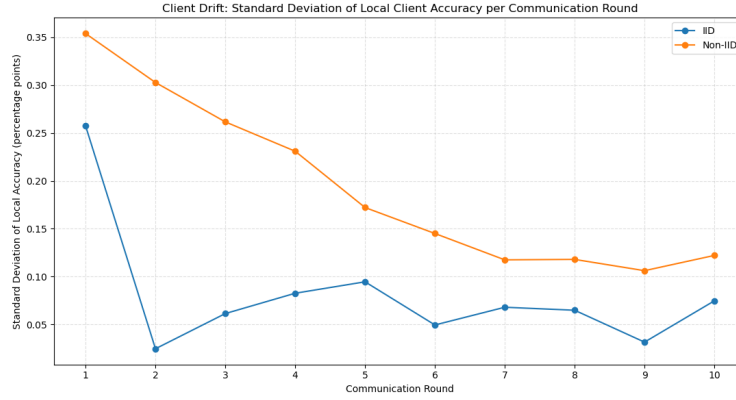


Figure 7.7: Standard deviation of local client accuracy per communication round, IID versus Non-IID.

The Non-IID standard deviation stays above the IID standard deviation in every round after the first, confirming that Non-IID clients disagree with each other substantially more than IID clients throughout training. This client-level disagreement is the underlying mechanism behind the 20.52 percentage-point global accuracy gap reported in Section 7.3.

7.5 Class-Wise Evaluation of the Final Global Models

Confusion matrices and per-class precision, recall and F1-score were computed for the final IID and Non-IID global models on the complete MNIST test set.

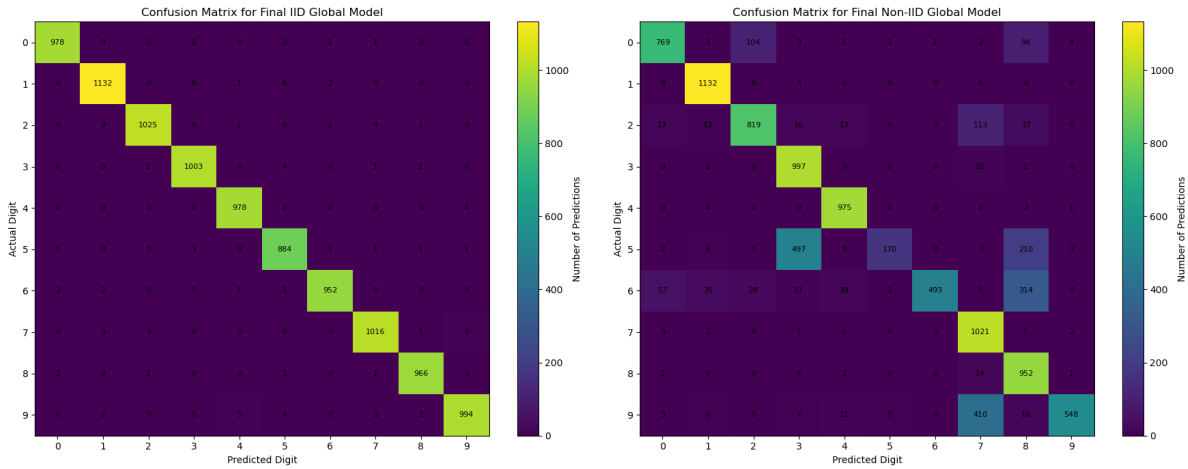


Figure 7.8: Confusion matrix for the final IID global model (left) and the final Non-IID global model (right).

Digit	IID Prec.	IID Rec.	IID F1	Non-IID Prec.	Non-IID Rec.	Non-IID F1
0	0.995	0.998	0.996	0.906	0.785	0.841
1	0.994	0.997	0.996	0.943	0.997	0.969
2	0.995	0.993	0.994	0.860	0.794	0.826
3	0.997	0.993	0.995	0.648	0.987	0.782
4	0.992	0.996	0.994	0.950	0.993	0.971
5	0.989	0.991	0.990	0.944	0.191	0.317
6	0.995	0.994	0.994	0.998	0.515	0.679
7	0.990	0.988	0.989	0.649	0.993	0.785

Digit	IID Prec.	IID Rec.	IID F1	Non-IID Prec.	Non-IID Rec.	Non-IID F1
8	0.994	0.992	0.993	0.585	0.977	0.732
9	0.987	0.985	0.986	0.980	0.543	0.699

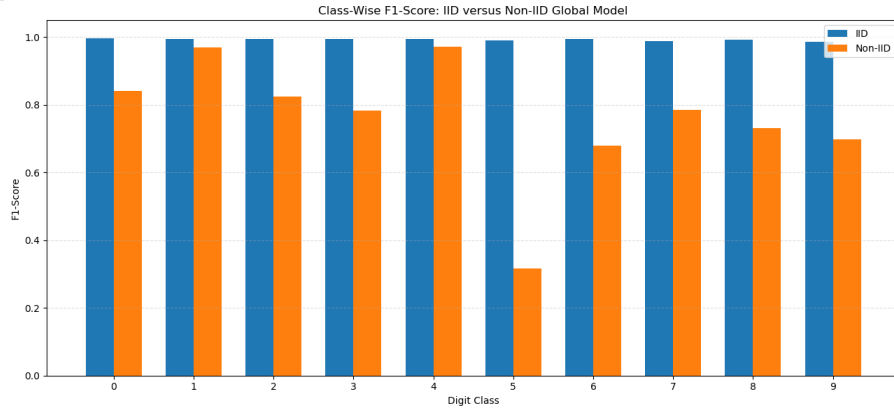


Figure 7.9: Class-wise F1-score, IID versus Non-IID global model.

Digit 5 shows the largest degradation, with its F1-score falling from 0.990 under IID training to 0.317 under Non-IID training, a drop of 0.673. Digits 6, 7, 8 and 9 also show substantial F1-score reductions (0.994 to 0.679, 0.989 to 0.785, 0.993 to 0.732 and 0.986 to 0.699 respectively), while digits 1 and 4 are comparatively unaffected. This uneven degradation matches the shard assignment in Section 3.2: classes that appear as a dominant class for only a single client, or that are split unevenly across clients with very different total representation, suffer the largest generalisation loss once local models are aggregated.

8. Consolidated Results

Experiment	Distribution	Local Epochs	Rounds	Initial Acc.	Final Acc.	Best Acc.	Final Loss
IID - Local Epoch 1	IID	1	10	96.57%	99.10%	99.17%	0.0286
IID - Local Epoch 2	IID	2	10	97.59%	99.28%	99.28%	0.0254
IID - Local Epoch 5	IID	5	10	98.36%	99.26%	99.30%	0.0317
Non-IID - Local Epoch 2	Non-IID	2	10	41.06%	78.76%	78.76%	0.6967

9. Analysis and Discussion

The IID configuration converges rapidly and stably. With two local epochs, global accuracy increases from 97.59% in Round 1 to 99.28% after ten rounds, with the lowest loss (0.0254) occurring in the final round. The local-epoch comparison produced final accuracies of 99.10%, 99.28% and 99.26% for 1, 2 and 5 local epochs respectively. Five local epochs reach the highest intermediate accuracy (99.30% at Round 9) but do not produce the strongest final result, despite requiring 2.5 times as much local computation per round as the two-epoch configuration; this is a direct illustration of diminishing returns from additional client-side computation under IID data.

The Non-IID model improves from 41.06% to only 78.76%, a result that remains 20.52 percentage points below the equivalent IID model despite identical architecture, initialisation, batch size and round count. Local client training accuracies remain near 100% in both settings, which demonstrates that strong client-level fit can coexist with weak global generalisation once label distributions are severely skewed: a client that fits its own two-shard subset almost perfectly is not thereby learning a decision boundary that generalises to the other eight digit classes.

Class-wise evaluation confirms that this degradation is concentrated in specific classes rather than spread evenly across all ten digits. Digit 5 is the most affected class, with its F1-score falling from 0.990 under IID training to 0.317 under Non-IID training. The round-wise client-accuracy dispersion (Section 7.4) provides the mechanistic explanation: Non-IID clients disagree with each other substantially more than IID clients throughout training, so their weighted average is a compromise between locally specialised models rather than a consensus update, and that compromise systematically under-serves the classes that are thinly or unevenly represented across clients.

9.1 Critical Analysis

Why does FedAvg perform differently on IID and Non-IID data?

IID clients contain similar class proportions, so their local objectives and gradient updates are broadly aligned; averaging these aligned updates produces a stable, effective global update, which is why the IID run reaches 99.28% global accuracy. Non-IID clients are dominated by different labels, so each local model specialises to its own narrow subset and produces updates that conflict with one another during aggregation. Although local training accuracy remains near 100% in the Non-IID setting, the global model reaches only 78.76%, which demonstrates that per-client fit and global generalisation are governed by different mechanisms in a federated system with heterogeneous data.

What is the impact of increasing the number of local epochs?

Increasing local epochs increases the amount of learning completed by each client before every communication event. In this experiment, five local epochs improved early-round accuracy (98.36% in Round 1, versus 97.59% at two epochs) but finished at 99.26%, marginally below the two-epoch result of 99.28%, while consuming 2.5 times as much local computation per round. Additional local epochs therefore reduce the number of communication rounds needed to reach a given accuracy level, but provide diminishing or even slightly negative returns on final accuracy, and under Non-IID data this additional local computation is expected to intensify client specialisation and worsen client drift rather than improve it.

Suggested improvement over standard FedAvg

FedProx is identified as the appropriate improvement over standard FedAvg for this setting. FedProx augments each client's local training objective with a proximal term that penalises large deviations from the current global weights. This constrains local specialisation directly, which is expected to reduce the client drift quantified in Section 7.4 and to narrow the IID-to-Non-IID accuracy gap observed in Section 7.3, particularly for the classes shown in Section 7.5 to be most affected by client heterogeneity. Implementing and evaluating FedProx experimentally is identified as future work and falls outside the defined scope of this assignment.

10. Limitations and Practical Considerations

This implementation demonstrates the FedAvg workflow under a controlled simulation rather than a production federated-learning deployment. The following limitations apply to the results presented in this report.

- All five clients are simulated sequentially within a single notebook environment rather than on physically separate systems.
- All clients participate in every communication round; client dropout and partial participation are not modelled.
- Communication latency, network failures, bandwidth consumption and model-transfer cost are not measured.
- All clients hold equal dataset sizes and identical computing capacity.
- The Non-IID distribution is intentionally severe and is constructed using label-based shards; real-world heterogeneity may be milder or structured differently.
- Only standard FedAvg is implemented and evaluated; FedProx is discussed as an improvement but not evaluated experimentally.
- Privacy-preserving mechanisms such as secure aggregation, encryption and differential privacy are outside the defined scope of this assignment.
- The experiment uses MNIST and a lightweight CNN; results may not transfer directly to larger, real-world medical-imaging datasets.

11. Conclusion

Federated Averaging was implemented manually for MNIST image classification across five simulated clients. Both IID and label-shard Non-IID partitions were evaluated without using TensorFlow Federated, Flower, FedML, PySyft or any other federated-learning framework.

Under IID data, the two-local-epoch configuration reached 99.28% global accuracy and 0.0254 loss after ten communication rounds. Increasing local epochs to five improved early-round learning but did not improve the final result, illustrating diminishing returns from additional client-side computation.

Under Non-IID data, final global accuracy reached only 78.76%, a 20.52 percentage-point gap relative to the equivalent IID run, even though every other experimental condition was held identical. Class-wise analysis showed substantial bias concentrated in specific digits, most severely digit 5, and the round-wise dispersion of client accuracy confirmed that this gap is driven by client drift: Non-IID clients disagree with each other throughout training far more than IID clients do.

These results demonstrate that standard FedAvg performs strongly when client data distributions are similar, but is sensitive to severe statistical heterogeneity across clients. FedProx and related drift-control methods are identified as the appropriate direction for improving stability under such heterogeneous conditions.

Execution Evidence

The accompanying Jupyter Notebook was executed from start to finish in Kubeflow. The excerpt below records representative runtime output from the primary IID experiment and the final notebook validation.

```
IID FEDAVG - LOCAL EPOCH 2
Rounds=10 LocalEpochs=2 BatchSize=32 Clients=5

Communication Round 1/10
Client_1 | Samples: 12000 | Local Accuracy: 97.09%
Client_2 | Samples: 12000 | Local Accuracy: 96.74%
Client_3 | Samples: 12000 | Local Accuracy: 97.03%
Client_4 | Samples: 12000 | Local Accuracy: 96.89%
Client_5 | Samples: 12000 | Local Accuracy: 96.45%
Global Loss: 0.0840 | Global Accuracy: 97.59%

Communication Round 10/10
Global Loss: 0.0254 | Global Accuracy: 99.28%

FINAL STATUS
Notebook validation status: All mandatory components are complete.
```

Figure 11.1: Representative execution output and final validation from the submitted notebook.

12. References

1. LeCun, Y., Cortes, C., and Burges, C. J. C. The MNIST Database of Handwritten Digits.
2. McMahan, H. B., Moore, E., Ramage, D., Hampson, S., and Aguera y Arcas, B. "Communication-Efficient Learning of Deep Networks from Decentralized Data." Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, 2017.
3. Li, T., Sahu, A. K., Talwalkar, A., and Smith, V. "Federated Optimization in Heterogeneous Networks." Proceedings of Machine Learning and Systems, 2020.
4. TensorFlow. TensorFlow and Keras Documentation.
5. NumPy Developers. NumPy Documentation.
6. Pandas Development Team. Pandas Documentation.
7. Matplotlib Development Team. Matplotlib Documentation.
8. scikit-learn Developers. scikit-learn Documentation.